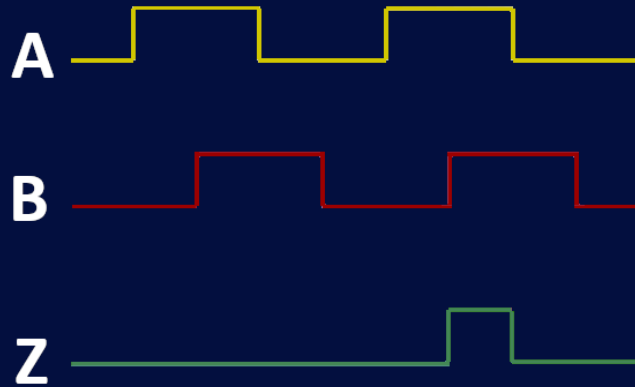


Incremental Encoders

How to emulate them with a Microcontroller



Leonardo Chieco

Incremental Encoders: How to Emulate Them with a Microcontroller

Incremental encoders are among the most widely used sensors in industrial automation systems, electric motor control, and motion systems. They make it possible to accurately measure the position, speed, and direction of rotation of mechanical shafts, providing essential information for the operation of drives, robots, automated machines, and CNC systems.

Their operating principle generally relies on the generation of two digital signals with a 90° phase shift (channels A and B). From these signals it is possible to determine both the direction of rotation and the movement resolution. In many cases, a third reference signal (Z or ZERO) is also present, which identifies a precise angular position.

During the development or testing of control systems that rely on encoders, it can be extremely useful to have a device capable of artificially simulating these signals without the need for an actual motor or encoder. In this tutorial we will therefore examine how an incremental encoder works and how its behavior can be emulated using an Arduino microcontroller, generating the A, B, and ZERO signals with precise timing through the hardware timers available on the device.

The result will be a simple yet effective encoder emulator, useful for testing, development, and experimentation in motion control systems.

Enjoy the reading!

About the Author

Leonardo Chieco is an electronic engineer with more than 20 years of experience in the design and development of automation control software (PC/PLC), the design of electronic boards for industrial applications, as well as firmware development, robotics, and mechatronics.

LinkedIn: <https://www.linkedin.com/in/leonardo-chieco-53550b129/>

Web: <https://leochieco.github.io/>

1. Introduction

An encoder is an electromechanical device used to convert a physical quantity related to motion (typically angular or linear position) into an electrical signal that can be interpreted by a control system. In practice, it allows the motion of a shaft or a carriage to be “translated” into digital or analog information, which is essential in automation, robotics, and motor control applications, such as systems based on brushless motors.

Encoders can generally be divided into two main categories: absolute encoders and incremental encoders. The former directly provide the unique position of the element at any given time, even after a power loss, thanks to an internal coding scheme (often multi-bit). The latter, instead, generate a sequence of pulses proportional to the displacement and therefore require an initial reference to determine the absolute position.

A further classification concerns the sensing technology used. Encoders can be optical (based on slotted disks and photodiodes), magnetic (using Hall-effect or magnetoresistive sensors), or capacitive.

Among incremental encoders, quadrature encoders are particularly widespread because, thanks to two phase-shifted signals, they allow not only the measurement of speed but also the determination of the direction of motion with high reliability.

Incremental encoders are widely used in industrial automation systems, in electric motor control, and in position and speed measurement systems. They typically generate two digital signals shifted by 90 electrical degrees (channels A and B), which make it possible to determine both the rotational speed and the direction of movement. Many encoders also provide a third reference signal, often referred to as Z or ZERO, which identifies a specific angular position.

During the commissioning of control systems, it can be very useful to have a tool capable of simulating the behavior of a real encoder. An encoder emulator makes it possible to generate A/B signals with a controlled and perfectly stable frequency, allowing hardware and software to be tested even in the absence of the motor or the physical encoder.

In this tutorial we will show how to build a programmable incremental encoder emulator using Arduino. As we will see, the proposed system is capable of generating quadrature signals (A and B) with adjustable frequency, as well as a ZERO reference signal synchronized with the pulse train. The frequency can be modified using push buttons, while a graphical OLED display shows the operating parameters of the device in real time.

By the end of this guide, we will know how to build a complete encoder emulator capable of generating stable and adjustable signals, useful both as an educational tool and as a practical aid in the development and testing of motion control systems.

2. Encoder in quadratura

A quadrature encoder is a rotary (or linear) transducer used to measure the angular position, speed, and direction of rotation of a mechanical shaft. It consists of a moving part (rotor) and a fixed part (stator) which, through an optical, magnetic, or capacitive system, generates periodic digital signals proportional to the motion.

The term “quadrature” refers to the fact that the two main signals generated by the encoder, called A and B, are shifted by 90 electrical degrees (that is, a quarter of a period).

This phase shift is the key feature that allows the direction of rotation to be determined:

- if B leads A, the rotation is counterclockwise;
- if A leads B, the rotation is clockwise, for example.;

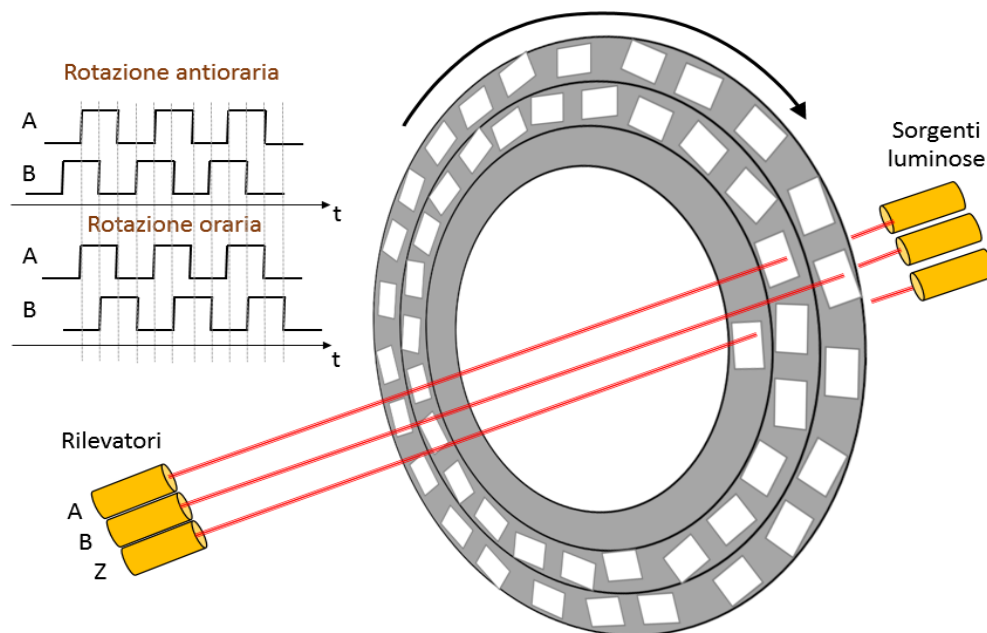


Figure 1: Principle diagram of a quadrature optical encoder

In general, a standard incremental encoder provides three digital signals:

- Signal A: a square wave whose frequency is proportional to the rotational speed.

- Signal B: another square wave, identical to A but shifted by 90° .

This phase shift allows the direction of motion to be determined.

Signal Z (or Index, or Zero pulse): a single pulse for each complete revolution.

It serves as an absolute reference: when the Z signal transitions from low to high, the shaft is at a known angular position (mechanical zero).

Counting the logical transitions of signals A and B allows the incremental angular position to be measured, while the Z pulse makes it possible to reset the counter or synchronize the position reference.

Let us assume that the encoder provides N pulses per revolution on channel A.

Since channel B also produces N pulses, but shifted by 90° , it is possible to obtain four useful edges for each complete cycle (rising and falling edges of both A and B).

This counting method, known as “x4 decoding”, multiplies the effective resolution by four. For example, with an encoder rated at 1000 pulses per revolution, it is possible to obtain 4000 steps per revolution.

The decoding logic (either hardware or software) observes the sequence of states of signals A and B:

- $00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$ (clockwise rotation)
- $00 \rightarrow 10 \rightarrow 11 \rightarrow 01 \rightarrow 00$ (counterclockwise rotation)

and updates the position counter accordingly.

In industrial applications and environments with strong electromagnetic interference, single-ended digital signals (TTL or CMOS) can degrade or become corrupted by noise, especially when transmitted over long cables.

To avoid these problems, professional encoders often use balanced differential transmission (e.g., the RS-422 standard). Each logical channel (A, B, Z) is therefore transmitted as a differential pair:

- A+ / A-
- B+ / B-

- $Z+ / Z-$

The receiving logic detects the logical state based on the voltage difference between the two wires:

- Logic level “1” $\rightarrow A+ > A-$
- Logic level “0” $\rightarrow A+ < A-$

In this way:

- common-mode noise (which couples equally to both conductors) is cancelled, since the receiver detects only the difference;
- higher immunity to electromagnetic noise is achieved, allowing stable transmission over long distances (even tens of meters);
- return currents on the cable shield are minimized.

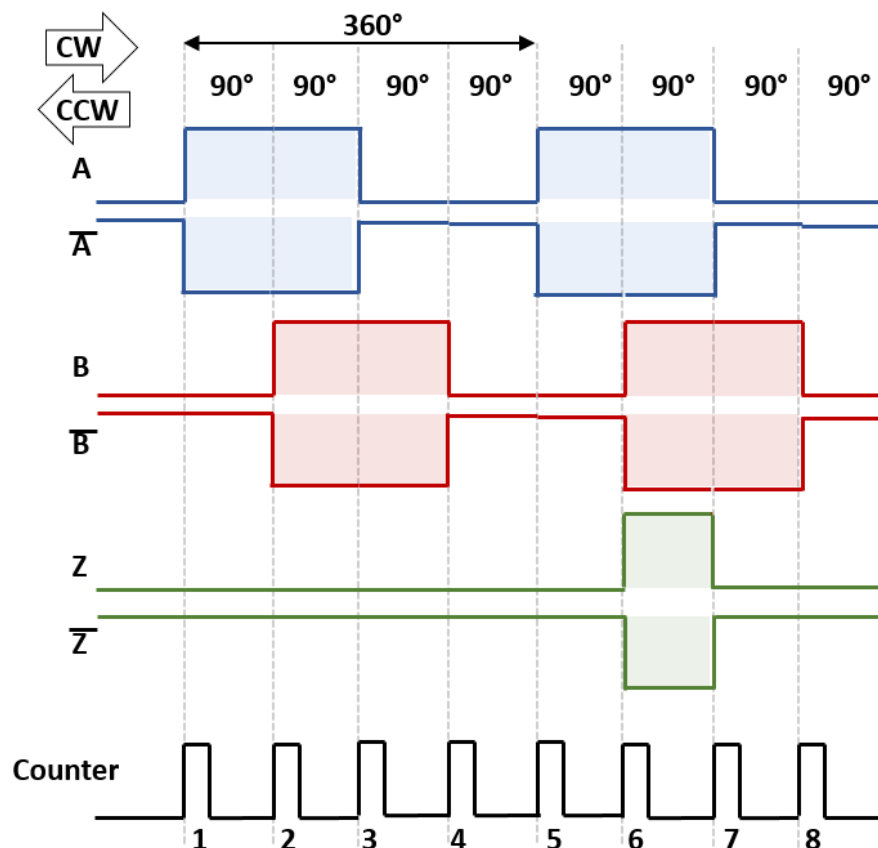


figure 2: Signals generated by a quadrature incremental encoder

When discussing quadrature signals, the first thing that comes to mind is rotary or linear encoders. However, this type of signal is also used in many other sensors and transducers that do not directly measure position but rather other physical quantities such as flow rate, pressure, speed, or light intensity.

The basic principle remains the same: generate two signals shifted by 90° , allowing both the magnitude of the phenomenon and the direction of variation to be determined.

Examples of sensors with quadrature outputs

- Flow sensors with magnetic or optical vanes: In some turbine flow meters (for gases or liquids), two photodiodes or magnetic sensors are positioned so that they produce pulses shifted by 90° . Quadrature processing makes it possible to determine the direction of the fluid flow (useful in bidirectional systems) and to improve the counting resolution.
- Speed sensors on toothed wheels (magnetic pick-ups): In tachometers or angular speed measurement systems (for example on motor shafts or transmission systems), two magnetoresistive or Hall sensors are placed so that they detect the passage of the teeth with a spatial offset equivalent to 90 electrical degrees. In this case as well, quadrature signals allow the direction of rotation to be determined and provide more accurate counting.

2.2 Operating Modes

As discussed earlier, a quadrature encoder provides two digital signals, A and B, shifted by 90 electrical degrees. These signals can be interpreted in three different ways by a control system (e.g., a PLC), directly affecting the resulting resolution.

X1 Mode

Only the rising edge of one channel (usually A) is counted.

For a complete cycle of signal A (one period), one countable pulse is generated.

This is the simplest mode, but it also provides the lowest resolution.

X2 Mode

Both the rising and falling edges of one channel are counted.

This doubles the number of pulses compared to X1, improving the counting resolution.

X4 Mode

All rising and falling edges of both channels A and B are counted.

Each complete cycle therefore produces four countable pulses, maximizing the resolution achievable from the encoder.

This mode provides the highest precision, but it requires slightly more sophisticated counting electronics.

Mode	Counted Edges	Counted Edges	Counted Edges
X1	Rising edge of channel A (or B) only	1×	Very high-speed applications
X2	Rising and falling edges of channel A (or B)	2×	Balanced resolution and frequency requirements
X4	All edges of both channels A and B	4×	Motion control and precision positioning

3. Emulator

A quadrature signal emulator is an extremely useful tool for engineers involved in the development, testing, or maintenance of systems that receive inputs from encoders or sensors with A/B/Z outputs. Such a device makes it possible to test control boards, PLCs, and data acquisition systems by simulating different operating conditions, such as rotation speed and direction. In this way, it becomes possible to verify the correct operation of counting algorithms, perform diagnostics on acquisition systems, and carry out maintenance and troubleshooting activities directly in the field.

When the goal is to generate quadrature signals at relatively high frequencies, a purely software-based approach often becomes insufficient, especially on platforms such as Arduino. Generating signal edges with the correct phase shift requires very precise timing, which is difficult to guarantee using software alone without introducing jitter or frequency limitations. For this reason, it is advisable to complement the microcontroller with a simple dedicated hardware logic stage.

In the project described in this tutorial, Arduino is mainly used as a clock generator. The microcontroller produces a square wave on two digital output pins, with an adjustable frequency up to approximately 10 kHz, which acts as the time base for the entire system. These two pins are also used to select the simulated direction of rotation. One pin generates the forward counting signal (clockwise direction), while the other enables counting in the reverse direction (counterclockwise).

These signals are sent to the UP and DOWN inputs of a binary counter, specifically the SN74193. The counter cyclically scans the binary states from 0 to 3, producing an ordered sequence of logical combinations at its outputs. The two least significant bits of the counter (Q_0 and Q_1) therefore form a two-bit cyclic encoding representing the four fundamental logic states associated with the A and B phases of an incremental encoder.

The Q_0 output of the counter already corresponds to signal B, while the XOR between Q_0 and Q_1 produces signal A.

With this simple combinational network, the signals Q_0 and Q_1 are mapped appropriately to obtain two digital outputs shifted by 90 electrical degrees, corresponding to the classic quadrature outputs A and B. The result is a pair of digital signals that faithfully reproduces the behavior of a real incremental encoder, in which the direction of motion is determined by the temporal order of the logic transitions of the two phases.

The resulting system is simple, inexpensive, and highly robust: the main timing is generated by Arduino, while the correct phase sequence is guaranteed by the hardware logic of the counter. This approach makes it possible to obtain stable quadrature signals even at relatively high frequencies.

State		Outputs	
Q_0	Q_1	$A = Q_0 \oplus Q_1$	$B = Q_1$
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	1

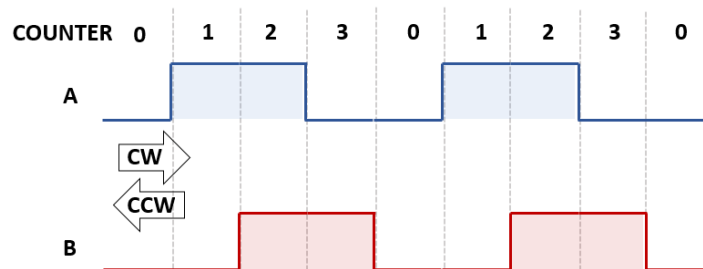


Figura 3: Counter state encoding

The two single-ended signals, A and B, are then applied to the inputs of a four-channel differential driver (model SN75192). This integrated circuit converts each TTL signal into a balanced differential pair ($A+/A-$ and $B+/B-$), generating symmetrical voltages with respect to the common reference.

The use of differential transmission, compliant with the RS-422 standard, provides greater immunity to electromagnetic interference and allows signals to be transmitted over long cables without signal degradation or false pulses.

In this way, using only a few inexpensive and readily available components, it is possible to build a digital encoder emulator fully compatible with the differential inputs of PLCs, motion controllers, or industrial measurement systems. Such a device is extremely useful for testing, calibration, and validation of firmware dedicated to quadrature signal counting.

The following figure shows the principle schematic of the emulator.

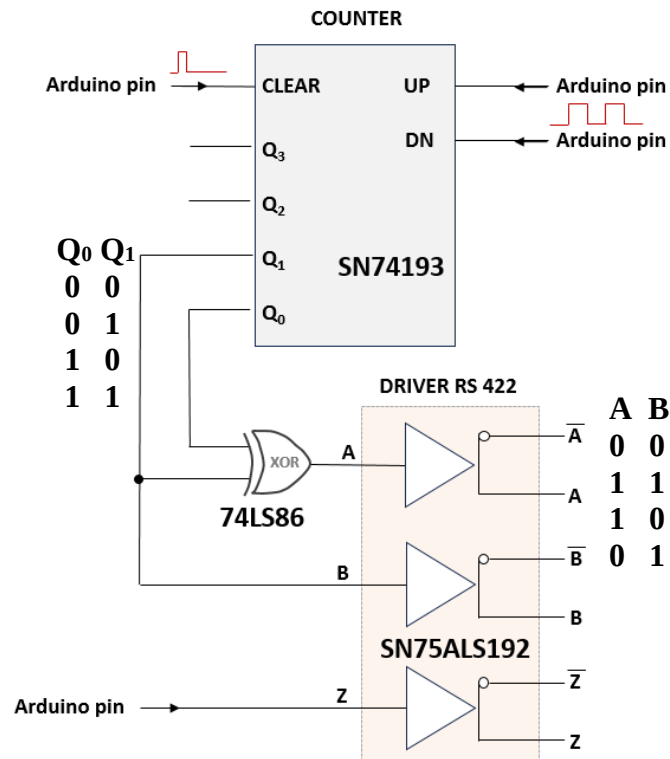


Figura 4: Schema di principio dell'emulatore di encoder

The microcontroller controls the emulator using only four signals:

- Clear: a pulse used to reset the outputs and start from a known configuration.
- UP: a pulse train used to count forward 00 → 01 → 11 → 10 → 00 (clockwise rotation).
- DN: a pulse train used to count backward 00 → 10 → 11 → 01 → 00 (counterclockwise rotation).
- Z: the “zero” (index) signal.

4. Generazione della frequenza base

Generating a square wave on a microcontroller like the Arduino Nano relies on the use of hardware timers, internal peripherals designed to count clock cycles and generate precisely timed events. Unlike software-based solutions (such as `delay()` or polling loops), which introduce timing errors and computational overhead, hardware timers provide stable and reproducible signals independent of the execution of the main code.

Specifically, Timer1 of the ATmega328P is a 16-bit counter that can operate in CTC mode (Clear Timer on Compare Match). In this configuration, the timer increments its value until it reaches the value set in the OCR1A register, then automatically resets to zero. If the automatic toggle mode of the OC1A pin (D9) is also enabled, the microcontroller toggles the pin's logic state on each compare match event, producing a perfectly symmetric square wave (50% duty cycle) without CPU intervention.

The signal frequency is determined by the relation:

$$f = \frac{f_{CPU}}{2 \cdot prescaler \cdot (OCR1A + 1)}$$

Where f_{CPU} is typically 16 MHz, and the prescaler is a selectable hardware divider (1, 8, 64, 256, 1024) that allows adjusting the timer's operating range.

Since the OCR1A register is 16-bit (maximum value 65535), it is not possible to cover the entire range from 1 Hz to 10 kHz with a single prescaler. Therefore, the prescaler must be dynamically selected according to the desired frequency. High frequencies require low prescalers to maintain good temporal resolution, while low frequencies require high prescalers to avoid counter overflow.

This approach allows extremely precise signal generation, with errors limited only to timer quantization (typically less than 0.1%) and virtually no jitter, making it ideal for applications such as digital clocks, modulation, embedded system testing, or incremental encoder simulation.

From an implementation standpoint, this configuration requires direct manipulation of the timer registers, avoiding Arduino library abstractions to maximize control and performance.

The following example shows a complete function that automatically sets the prescaler and compare value to generate a square wave on D9 in the 1 Hz – 10 kHz range:

```
#define PIN_OUT 9 // OC1A: Timer1 hardware output pin

void setupTimer1(uint32_t freq) {

  pinMode(PIN_OUT, OUTPUT); // configure the pin as digital output

  // ===== TIMER RESET =====
  TCCR1A = 0; // clear Timer1 control register A
  TCCR1B = 0; // clear Timer1 control register B

  // ===== HARDWARE OUTPUT =====
  // COM1A0 = 1 → automatic toggle of OC1A pin (D9)
  // the pin changes state at every compare match
  TCCR1A |= (1 << COM1A0);

  // ===== TIMER MODE =====
  // WGM12 = 1 → CTC mode (Clear Timer on Compare Match)
  // the timer counter is reset when it reaches OCR1A
  TCCR1B |= (1 << WGM12);

  uint16_t prescalerBits;
  uint32_t prescaler;

  // ===== PRESCALER SELECTION =====
  // This allows the generator to cover a wide frequency range
  //
  // | CS12 | CS11 | CS10 | Prescaler |
  // | ---- | ---- | ---- | - - - - - |
  // | 0    | 0    | 1    | 1          |
  // | 0    | 1    | 0    | 8          |
  // | 0    | 1    | 1    | 64         |
  // | 1    | 0    | 0    | 256        |
  // | 1    | 0    | 1    | 1024       |
  //
  if (freq >= 2000) {
    prescaler = 1;
    prescalerBits = (1 << CS10); // prescaler = 1
  }
  else if (freq >= 200) {
    prescaler = 8;
```

```

prescalerBits = (1 << CS11); // prescaler = 8
}
else if (freq >= 30) {
prescaler = 64;
prescalerBits = (1 << CS11) | (1 << CS10); // prescaler = 64
}
else if (freq >= 4) {
prescaler = 256;
prescalerBits = (1 << CS12); // prescaler = 256
}
else {
prescaler = 1024;
prescalerBits = (1 << CS12) | (1 << CS10); // prescaler = 1024
}

// ===== OCR1A CALCULATION =====
// Frequency formula:
//
// f = F_CPU / (2 * prescaler * (OCR1A + 1))
//
// solving for OCR1A:
uint32_t ocr = (16000000UL / (2UL * prescaler * freq)) - 1;

OCR1A = ocr; // load compare value into the register

// ===== TIMER START =====
TCCR1B |= prescalerBits; // start the timer with the selected prescaler
}

void setup() {

setupTimer1(1); // example: generate a 1Hz signal

}

void loop() {

// nothing here: the signal is generated entirely in hardware

}

```

This code implements a fully hardware-based signal generation, ensuring maximum stability and freeing the microcontroller for other tasks such as user interface management, communication, and control.

An important aspect to consider is that the duty cycle is fixed at 50% in toggle mode; for applications requiring a variable duty cycle, it is necessary to use PWM modes (Fast PWM or Phase Correct). Additionally, the ultimate accuracy depends on the tolerance of the microcontroller's crystal or internal oscillator.

5. Arduino Software

We now have all the elements to define the software specifications for the emulator:

- The entire system is controlled by an Arduino-family microcontroller, responsible for timed signal generation, user interface management, and display updates.
- The frequency of the generated pulse train is adjustable by the user via push buttons, allowing continuous variation from 1 Hz to 10 kHz.
- The selected frequency is displayed on a graphical OLED display, along with the logical state of the ZERO output.
- The pulse train does not start immediately when the device is powered on; it begins only after performing a counter reset sequence and pressing the start button. This ensures proper synchronization of the test system connected to the emulator.
- The system's timing operation relies on the Timer1 hardware of the microcontroller, configured in CTC mode (Clear Timer on Compare Match). In this configuration, the timer generates a periodic interrupt when the counter reaches the value set in the compare register. The interrupt associated with the vector `TIMER1_COMPA_vect` is used to generate the ZERO signal synchronized with the pulse sequence, remaining active for a duration equal to half the period of the generated frequency, thus ensuring temporal consistency with the produced pulse train.
- To improve user interface readability, the software handles the display of the ZERO state separately from the physical signal generated on the digital pin. Specifically, when the ZERO output goes high, the display shows the text "ON". This indication remains visible for at least 500 ms, even if the actual pulse is shorter. If the ZERO output remains active longer than 500 ms (for example, at very low frequencies), the ON text remains displayed for the entire duration of the pulse. This logic prevents graphical flickering and ensures that the user can correctly perceive the event even at high frequencies.

The software I prepared, provided in the following pages, is structured modularly to allow future improvements.

6. Schematic

The electrical schematic of the emulator is relatively simple. The heart of the system is an Arduino Nano, which manages the overall device operation and drives a standard OLED display with I²C interface used to show operational parameters.

Five push buttons allow the user to access all the main system functions, specifically:

- Setting the base operating frequency (UP and DOWN);
- Starting and stopping the pulse generation (START/STOP);
- Selecting the quadrature sequence direction (DIR);
- Generating the zero signal (ZERO).

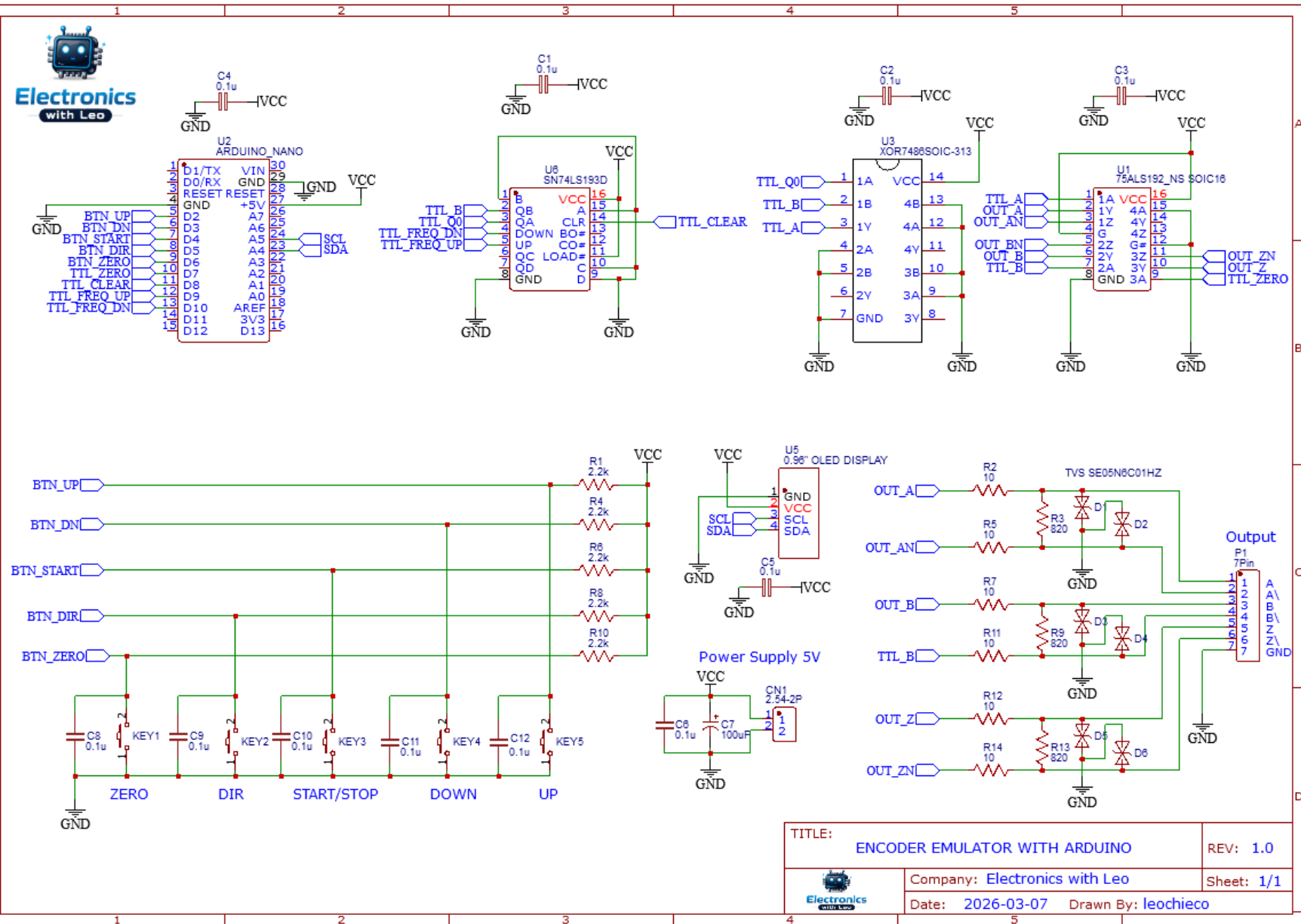
The generation of logical sequences is handled by a SN74193 counter, which cyclically steps through the four states required for quadrature counting, as described previously. The A signal is obtained via an XOR gate in a 7486 IC, which appropriately processes the counter outputs.

Next, a 75ALS192 converts the A, B, and Z signals from single-ended to differential, making them compatible with the standard interfaces used in industrial encoders.

All outputs are protected by TVS diodes to enhance system robustness against overvoltage or environmental disturbances.

Naturally, this is a basic schematic, which can be further expanded and improved in several ways. For example, galvanic isolation could be introduced between the CPU (Arduino) and the field, increasing both immunity to interference and overall system safety.

In this tutorial, however, the goal is simply to provide a foundation for future experimentation and development.



7. Source code

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ===== OLED =====
Adafruit_SSD1306 display(128,64,&Wire,-1);

// ===== PIN =====
#define BTN_UP      2
#define BTN_DOWN    3
#define BTN_START   4
#define BTN_DIR     5
#define BTN_ZERO    6

#define PIN_OC1A    9
#define PIN_OC1B    10

#define PIN_ZERO    7
#define PIN_RESET   8

// ===== PARAMETERS =====
uint32_t frequency = 2;
bool directionForward = true;

const uint16_t debounceTime = 20;

// ===== SYSTEM STATE =====
enum RunState {STOPPED, RESETTING, RUNNING};
RunState runState = STOPPED;

uint32_t resetStartTime = 0;

// ===== ZERO =====
volatile bool triggerZero=false;
volatile uint32_t zeroOffTime=0;

bool zeroOnDisplay=false;
uint32_t zeroDisplayTimer=0;

// ===== BUTTON STRUCT =====
struct Button{
  uint8_t pin;
  bool lastState;
  uint32_t pressTime;
  uint32_t lastRepeat;
  bool shortHandled;
};

Button btnUp    ={BTN_UP,HIGH,0,0,false};
Button btnDown  ={BTN_DOWN,HIGH,0,0,false};
```

```

// ===== DEBOUNCE VAR =====
bool dirState=HIGH;
bool lastDirReading=HIGH;
uint32_t lastDirDebounce=0;

bool startState=HIGH;
bool lastStartReading=HIGH;
uint32_t lastStartDebounce=0;

bool zeroState=HIGH;
bool lastZeroReading=HIGH;
uint32_t lastZeroDebounce=0;

// ===== TIMER =====
void stopTimer()
{
    TCCR1A=0;
    TCCR1B=0;
}

void setupTimer1(uint32_t freq)
{
    TCCR1A=0;
    TCCR1B=0;

    pinMode(PIN_OC1A, OUTPUT);
    pinMode(PIN_OC1B, OUTPUT);

    if(directionForward)
        TCCR1A|=(1<<COM1A0);
    else
        TCCR1A|=(1<<COM1B0);

    TCCR1B|=(1<<WGM12);

    uint32_t prescaler;
    uint16_t bits;

    if(freq>=2000){prescaler=1;bits=(1<<CS10);}
    else if(freq>=200){prescaler=8;bits=(1<<CS11);}
    else if(freq>=30){prescaler=64;bits=(1<<CS11)|(1<<CS10);}
    else if(freq>=4){prescaler=256;bits=(1<<CS12);}
    else {prescaler=1024;bits=(1<<CS12)|(1<<CS10);}

    uint32_t ocr=(16000000UL/(2UL*prescaler*freq))-1;

    OCR1A=ocr;
    OCR1B=ocr;

    TIMSK1=(1<<OCIE1A);

    TCCR1B|=bits;
}

```

```

// ===== TIMER ISR =====
ISR(TIMER1_COMPA_vect)
{
    if(triggerZero)
    {
        digitalWrite(PIN_ZERO, HIGH);

        zeroOffTime=micros()+(500000UL/frequency);

        triggerZero=false;
    }

    if(zeroOffTime && micros()>=zeroOffTime)
    {
        digitalWrite(PIN_ZERO, LOW);
        zeroOffTime=0;
    }
}

// ===== DISPLAY =====
void drawUI()
{
    display.clearDisplay();

    display.drawRect(0, 2, 128, 62, SSD1306_WHITE);

    display.setTextSize(1);

    int16_t x1, y1; uint16_t w, h;
    display.getTextBounds("ENCODER-EMU", 0, 0, &x1, &y1, &w, &h);

    display.fillRect((128-w)/2-2, 0, w+4, h, SSD1306_BLACK);

    display.setCursor((128-w)/2, 0);
    display.print("ENCODER-EMU");

    display.setCursor(4, 18);
    display.print("Freq:");
    display.print(frequency);
    display.print("Hz");

    display.setCursor(4, 32);
    display.print("Dir:");
    display.print(directionForward?"Forward":"Backward");

    display.setCursor(4, 46);
    display.print("State:");
    display.print(runState==RUNNING?"RUN":"STOP");

    display.setCursor(80, 46);
    display.print("Z:");
    display.print(zeroOnDisplay?"ON":"OFF");
}

```

```

    display.display();
}

// ===== STEP =====
uint16_t stepValue(uint32_t t)
{
    if(t<500)return 1;
    else if(t<1500)return 10;
    else if(t<3000)return 100;
    else return 1000;
}

// ===== FREQ BUTTON =====
void handleFreqButton(Button &b,bool increase)
{
    bool state=digitalRead(b.pin);

    if(state==LOW && b.lastState==HIGH)
    {
        b.pressTime=millis();
        b.lastRepeat=millis();
        b.shortHandled=false;
    }

    if(state==LOW)
    {
        uint32_t t=millis()-b.pressTime;

        if(t<500 && !b.shortHandled)
        {
            if(increase)frequency++;
            else if(frequency>1)frequency--;

            b.shortHandled=true;
        }

        else if(t>=500)
        {
            if(millis()-b.lastRepeat>500)
            {
                uint16_t step=stepValue(t);

                if(increase)frequency+=step;
                else if(frequency>step)frequency-=step;

                if(frequency>10000)frequency=10000;

                b.lastRepeat=millis();
            }
        }

        if(runState==RUNNING)

```

```

        setupTimer1(frequency);
    }

    b.lastState=state;
}

// ===== DIR BUTTON =====
void handleDirButton()
{
    bool reading=digitalRead(BTN_DIR);

    if(reading!=lastDirReading)
        lastDirDebounce=millis();

    if(millis()-lastDirDebounce>debounceTime)
    {
        if(reading!=dirState)
        {
            dirState=reading;

            if(dirState==LOW)
            {
                directionForward=!directionForward;

                if(runState==RUNNING)
                    setupTimer1(frequency);
            }
        }
    }

    lastDirReading=reading;
}

// ===== ZERO BUTTON =====
void handleZeroButton()
{
    bool reading=digitalRead(BTN_ZERO);

    if(reading!=lastZeroReading)
        lastZeroDebounce=millis();

    if(millis()-lastZeroDebounce>debounceTime)
    {
        if(reading!=zeroState)
        {
            zeroState=reading;

            if(zeroState==LOW)
            {
                triggerZero=true;

                zeroOnDisplay=true;
                zeroDisplayTimer=millis();
            }
        }
    }
}

```

```

    }
}

lastZeroReading=reading;

if(zeroOnDisplay)
{
    if(!digitalRead(PIN_ZERO) && millis()-zeroDisplayTimer>=500)
        zeroOnDisplay=false;
}
}

// ===== START BUTTON =====
void handleStartButton()
{
    bool reading=digitalRead(BTN_START);

    if(reading!=lastStartReading)
        lastStartDebounce=millis();

    if(millis()-lastStartDebounce>debounceTime)
    {
        if(reading!=startState)
        {
            startState=reading;

            if(startState==LOW)
            {
                if(runState==RUNNING)
                {
                    stopTimer();
                    runState=STOPPED;
                }
                else if(runState==STOPPED)
                {
                    stopTimer();

                    digitalWrite(PIN_RESET,HIGH);

                    resetStartTime=millis();

                    runState=RESETTING;
                }
            }
        }
    }

    lastStartReading=reading;
}

// ===== RESET SEQUENCE =====
void handleResetSequence()

```

```

{
  if(runState==RESETTING)
  {
    if(millis()-resetStartTime>=100)
    {
      digitalWrite(PIN_RESET,LOW);

      setupTimer1(frequency);

      runState=RUNNING;
    }
  }
}

// ===== SETUP =====
void setup()
{
  pinMode(BTN_UP,INPUT_PULLUP);
  pinMode(BTN_DOWN,INPUT_PULLUP);
  pinMode(BTN_START,INPUT_PULLUP);
  pinMode(BTN_DIR,INPUT_PULLUP);
  pinMode(BTN_ZERO,INPUT_PULLUP);

  pinMode(PIN_ZERO,OUTPUT);
  pinMode(PIN_RESET,OUTPUT);

  digitalWrite(PIN_ZERO,LOW);

  display.begin(SSD1306_SWITCHCAPVCC,0x3C);
  display.setTextColor(SSD1306_WHITE);

  digitalWrite(PIN_RESET,HIGH);
  delay(100);
  digitalWrite(PIN_RESET,LOW);

  drawUI();
}

// ===== LOOP =====
void loop()
{
  handleFreqButton(btnUp,true);
  handleFreqButton(btnDown,false);

  handleDirButton();
  handleZeroButton();
  handleStartButton();

  handleResetSequence();

  drawUI();
}

```